DUMMY INTERCOM · ACCEPTANCE

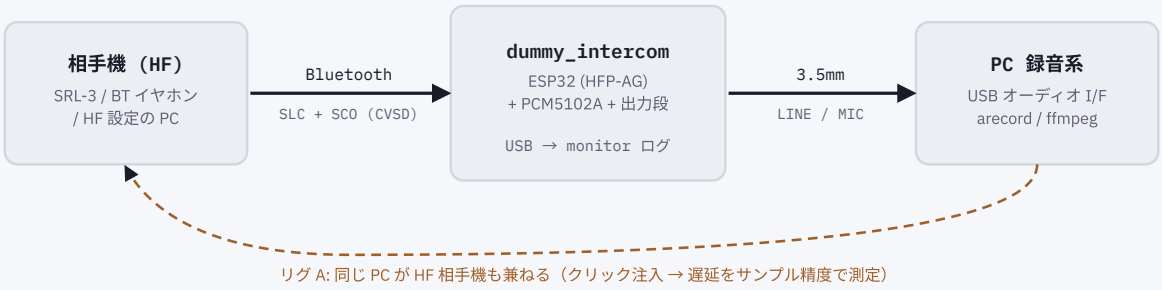
製品チェック方法 — 受け入れ試験手順書

受信専用インカムブリッジ（ESP32-WROOM-32 + PCM5102A、HFP-AG / SCO-CVSD）の組み立て機を、Bluetooth 相手機と PC 録音系で検証するための手順書。仕様書 `dummy_intercom_spec.md` rev 0.6 §11 の T1〜T13 に測定系を付けて具体化したもの。

0 試験の考え方と 3 つのリグ

> **PIXEL 8 PRO は音声ソースにならない**
>
> 本機は HFP **Audio Gateway**（電話役）。スマホも AG なので **AG** 同士では音声（**HFP/SCO**）が接続されない。Pixel 8 Pro は ペアリング・ディスカバリ・CoD 表示・「AG のみ提示」の負検査（T2b）にだけ使う。**3.5 mm** から音は出ない。
>
> 音声を通す相手機は HFP **Hands-Free**（**HF**）役に限る → SENA SRL-3、Bluetooth イヤホン/ヘッドセット、または HF 設定にした Linux PC。

リグ	相手機（BT）	用途	担当する試験
A	Linux PC を HFP-HF 化（同一 PC で信号注入＋録音）	測定系。テスト信号を注入し、同じ PC で 3.5 mm 出力を録音 → サンプル精度で比較	T3 T5 T6 T7 T9 T13 T14
B	SENA SRL-3 実機	受け入れ系。実運用そのもの	T3 T4 T8 T10 T11 T12 T13 + R1 確認
C	Bluetooth イヤホン/ヘッドセット	簡易ベンチ。SCO 音声経路（recv→RB→I2S→DAC→出力段）の切り分け。SRL-3 が無いときの代用	T3 T7 T9 T14（音声内容以外）
—	Google Pixel 8 Pro	ペアリング / ディスカバリ / CoD / 負検査のみ	T1 T2a T2b T12



相手機で音声を送り（リグ A は PC 自身、B は SRL-3 のインカム、C はイヤホンのマイク）、dummy_intercom の 3.5 mm 出力を PC で録って評価する。

---

1 準備するもの

- 組み立て済み dummy_intercom 機（ファーム書き込み済み。`esp32-flashing-guide.pdf` 参照）
- USB ケーブル（ESP32 の `monitor` ログ取得用）＋ 5 V モバイルバッテリー（電源系の試験用）
- 相手機：SENA SRL-3、および/または BT イヤホン、および/または Bluetooth アダプタ付き Linux PC
- **PC** 録音系：ライン入力のある USB オーディオインターフェース（LINE モードは 0.3–1.0 Vrms、PC のマイク入力では過大）。Debian 機なら `arecord` / `pw-record` / `ffmpeg` は導入済み
- テスター（出力 Vrms 実測用）、可能ならオシロ（T5 補助）
- Pixel 8 Pro（T1 / T2）
- 別 Linux PC + USB BT ドングル + `btmon`（T13。リグ A の PC と兼用可）

---

2 試験環境のセットアップ

2.1 dummy_intercom 側（全リグ共通）

```
# ログを常時取得（タイムスタンプ付き）。試験中は流しっぱなし
idf.py -p /dev/ttyUSB0 monitor | tee qa_$(date +%Y%m%d_%H%M).log
```

- 試験開始前に **BOND-CLR（GPIO14）3 秒長押し** → `factory reset ... erased` → 再起動。ボンド無し＝discoverable であること。
- 起動ログに `HFP-AG ready, name="DummyIntercom" (discoverable)` を確認。

- 出力段のジャンパを試験対象のモード（LINE か MIC）に設定。レベル半固定 VR は中央から開始。

2.2 リグ A — Linux PC を HFP-HF 化

```
# ペアリング & 接続（PC が HF 役で AG=ESP32 に接続）
bluetoothctl
  power on
  scan on          # DummyIntercom の MAC を確認
  scan off
  pair <MAC>       # PIN 不要（Just Works）
  trust <MAC>
  connect <MAC>
  quit

# HFP プロファイルと sink を確認
pactl set-card-profile bluez_card.<MAC> headset-audio-gateway
pactl list short sinks   | grep bluez     # 注入先: bluez_output.<MAC>.headset-audio-gateway
pactl list short sources | grep bluez
```

ESP32 ログに `SLC connected` → `SCO connected` → `codec negotiation result: 2`（CVSD）が出れば接続確立。

2.3 リグ B — SENA SRL-3

1. SRL-3 を取扱説明書に従い「電話（**Phone**）ペアリング」モードにする。
2. SRL-3 側で `DummyIntercom` を選択（PIN 不要）。ログに `SLC connected` / `peer BDA saved`。
3. SRL-3 でユニバーサルインターコム（またはインカム通話）を開始 → `SCO connected`。

2.4 リグ C — Bluetooth イヤホン

イヤホンをペアリングモードにして起動。ESP32 は discoverable。イヤホンが接続後、AG 側 keepalive が 3 秒周期で `audio_connect` を打つので SCO が張られる。イヤホンのマイクに向かって話す → **3.5 mm** に出る。挙動はイヤホン機種で差がある（SCO を拒否する個体あり）。

2.5 btmon 併走（T13）

```
sudo btmon | tee qa_hci_$(date +%Y%m%d_%H%M).log
```

2.6 PC 録音系のキャリブレーション

```
# 録音デバイス一覧
arecord -l
# 既知電圧（例: 発振器で 0.316 Vrms = -10 dBu 相当）を入れて録音し dBFS を測る
arecord -D hw:1,0 -f S16_LE -r 48000 -c 1 -d 5 cal.wav
ffmpeg -i cal.wav -af volumedetect -f null -   # mean_volume / max_volume (dBFS)
# → 「X Vrms = Y dBFS」の換算係数を記録。以降の T7/T9 はこの係数で Vrms 換算
```

---

3 試験項目一覧

ID	目的	手順（要旨）	合格基準	リグ
T1	起動・ディスカバリ	電源投入。Pixel の Bluetooth 画面を開く	10 秒以内に `DummyIntercom` が一覧表示。電話アイコン（CoD 0x60020C）。ログに `HFP-AG ready`	Pixel
T2a	ペアリング	SRL-3 の電話ペアリングから接続。Pixel からもペア設定	PIN 入力なしで成功。ログ `SLC connected`（SRL-3）/ `peer BDA saved`。Pixel は「ペア済み」	B / Pixel
T2b	負検査（AG のみ提示）	Pixel とペア後、通話音声・メディア音声の接続状態を見る	音声プロファイルは接続されない（グレーアウト等）。`SLC connected` は出ない。これが正常	Pixel
T3	SLC / SCO 確立	相手機で音声リンクを開始（B: インカム / A: プロファイル接続）	3 秒以内に `SCO connected`、SCO LED 点灯、3.5 mm に音声。`codec negotiation result: 2`（CVSD）	A / B / C
T4	音声疎通・了解度	2 台目インカムから通話 + 装着者も発話。3.5 mm を録音	音声が識別できる。グループ音声が **HFP** に乗るかを RMS ログと録音で記録（R1）	B
T5	エンドツーエンド遅延	リグ A：HFP sink にクリック列を注入しつつ 3.5 mm 出力を同 PC で録音。相互相関で遅延を算出	< 100 ms（可能なら < 60 ms）。測定間の変動が小さい（一定オフセット）	A
T6	連続性・ドロップアウト	30 分連続で正弦波を流し録音。無音区間・不連続を検出	3 秒以上の無音・断が発生しない。アンダーランログの急増なし	A / B
T7	出力レベル	1 kHz（帯域内 300–3400 Hz）フルスケール相当を流し、J1 出力をテスター実測 + 録音で確認	LINE: 0.3–1.0 Vrms / MIC: 3–30 mVrms。VR で調整範囲に収まる	A / C

ID	目的	手順（要旨）	合格基準	リグ
T8	自動再接続	SRL-3 電源断→復帰。距離を離す→戻す（各 ×3）	手動操作なしで再接続。LINK / SCO 復帰。ログに `reconnect slc_connect` → `SLC connected`	B
T9	無音時ノイズフロア	SCO 確立・相手機は無音。3.5 mm を 60 秒録音し dBFS 測定	ハム・ノイズが -60 dBFS 相当以下（絶縁トランス使用時）。50/60 Hz とその倍音が突出しない	A / C
T10	電源断耐性	動作中にモバイルバッテリーを 10 回遮断	毎回正常起動。NVS 破損なし（`NVS needs erase` が出ない）。再ペアリング不要	B
T11	カメラ連携	J1（MIC モード）→ Insta360 マイクアダプター → X4 で録画	X4 録画に音声が記録され、レベルが適正（クリップ/過小でない）	B
T12	ボンディング消去	BOND-CLR 3 秒長押し	ログ `factory reset ... erased` → 再起動 → 再び discoverable、初回ペアリング可能	B / Pixel
T13	コーデック互換	btmon で SLC シーケンスを記録	`AT+BRSF` / `AT+BAC` / `+BCS: 1` / `Setup Synchronous Connection Complete (status 0x00)`。SRL-3 が CVSD を受容。`mSBC negotiated unexpectedly` の警告が出ない	A / B
T14	LED 状態	各フェーズで 3 LED を目視	PWR：起動で点灯。LINK：未接続=消灯 / 再接続中=1 Hz 点滅 / 接続=点灯。SCO：音声なし=消灯 / 確立=点灯 / バッファ異常=速い点滅	全
T15	グラウンドループ	USB 給電（PC）と バッテリー給電で T9 を比較。絶縁トランス有無でも比較	バッテリー + 絶縁トランスでハムが最小。USB 給電時の増加が許容範囲	A

4 測定方法の詳細

4.1 レベル（T7）／ノイズフロア（T9）

```
# 60 秒録音（インターフェースは hw:1,0 の例、48 kHz mono）
arecord -D hw:1,0 -f S16_LE -r 48000 -c 1 -d 60 t9_silence.wav

# RMS / ピークを dBFS で
ffmpeg -i t9_silence.wav -af astats=metadata=1 -f null -   2>&1 | grep -E 'RMS
level|Peak level'
ffmpeg -i t9_silence.wav -af volumedetect -f null -        2>&1 | grep -E
'mean_volume|max_volume'

# ハム成分の確認（スペクトル画像を書き出し、50/60 Hz を目視）
ffmpeg -i t9_silence.wav -lavfi showspectrumpic=s=1024x512 t9_spectrum.png
```

キャリブレーション係数（§2.6）で dBFS → Vrms 換算。T7 は J1 をテスターの AC レンジでも実測し、両者が一致することを確認。

4.2 エンドツーエンド遅延（T5）

リグ A で、注入と録音を同一 PC で行い時間基準を共有する。1 秒間隔のクリック（インパルス）を HFP sink に流しながら 3.5 mm 出力を録音し、両波形の相互相関ピーク位置の差＝遅延。

```
# クリック生成（10 発、1 秒間隔）
ffmpeg -f lavfi -i "sine=f=1000:d=0.005" -af "apad=pad_dur=0.995" -stream_loop 9
clicks.wav

# 端末1: 録音開始 / 端末2: 注入
arecord -D hw:1,0 -f S16_LE -r 48000 -c 1 -d 15 t5_out.wav
paplay -d bluez_output.<MAC>.headset-audio-gateway clicks.wav

# 遅延算出（numpy）。無ければ Audacity で 2 波形を並べて目視でも可
python3 - <<'PY'
import wave, numpy as np
def rd(f):
    w=wave.open(f);
d=np.frombuffer(w.readframes(w.getnframes()),'<i2').astype(float); return d,
w.getframerate()
ref,fs = rd('clicks.wav'); out,_ = rd('t5_out.wav')
n=min(len(ref),len(out)); c=np.correlate(out[:n]-out[:n].mean(), ref[:n]-
ref[:n].mean(),'full')
lag=(c.argmax()-(n-1))/fs
print(f'end-to-end delay ~ {lag*1000:.1f} ms')
PY
```

3 回以上測り、平均と最大−最小（ジッタ）を記録。`app_config.h` の `JITTER_TARGET_BYTES`（60 ms）に対し妥当か確認。

4.3 ドロップアウト（T6）

```
# 30 分、1 kHz 正弦を流しつつ録音
ffmpeg -f lavfi -i "sine=f=1000:d=1800" t6_tone.wav
arecord -D hw:1,0 -f S16_LE -r 48000 -c 1 -d 1810 t6_out.wav
# 無音区間（-50 dB 以下が 0.2 秒以上）を列挙
ffmpeg -i t6_out.wav -af silencedetect=n=-50dB:d=0.2 -f null - 2>&1 | grep
silence_
```

同時に ESP32 ログの `audio_i2s` アンダーラン/オーバーラン数を集計。ISS-015（クロックドリフト）の兆候＝時間に比例した断の増加。

4.4 コーデック互換（T13）

btmon ログで SLC 確立シーケンスを追う。確認する行：

- `AT+BRSF=` / `+BRSF:`（feature 交換）
- `AT+BAC=1`（HF が CVSD のみ提示）／ `+BCS: 1`（AG が CVSD 選択）／ `AT+BCS=1`（HF 確認）
- `HCI Setup Synchronous Connection` → `Synchronous Connection Complete` の `Status: Success (0x00)`、`Air Mode: CVSD`
- SCO パケットが継続して流れる（途中で止まらない）

---

5 記録シート

ID	項目	実測値 / 観測	合否	日付 / 担当
T1	起動・ディスカバリ			
T2a	ペアリング（SRL-3 / Pixel）			
T2b	負検査（AG のみ提示）			
T3	SCO 確立（CVSD、3 秒以内）			
T4	音声疎通・グループ音声（R1）			
T5	遅延（平均 / ジッタ）	ms / ms		
T6	30 分連続・断の有無			
T7	出力レベル（Vrms、モード）	Vrms（LINE / MIC）		

ID	項目	実測値 / 観測	合否	日付 / 担当
T8	自動再接続			
T9	ノイズフロア（dBFS / Vrms）	dBFS		
T10	電源断 ×10・NVS			
T11	カメラ連携（X4 録音）			
T12	ボンディング消去			
T13	コーデック（btmon）			
T14	LED 3 状態			
T15	グラウンドループ比較			

ファーム版数（`git rev-parse --short HEAD`／未 commit なら日付）、IDF 版数、相手機の型番・ファーム版数、出力段モードとジャンパ設定、絶縁トランス有無、給電方法を各シートに併記する。

---

6 既知の注意点

- **R1**（最重要）：SRL-3 がユニバーサルインターコム音声を HFP に流すかは未確認。T4 で `bt_data_path` の RMS ログと録音を突き合わせ、誰の声が乗っているか（装着者のみ / グループ全員）を記録する。最悪でも装着者マイク音声は取得できる想定。
- **ISS-008 / 012**：ESP-IDF HFP-AG + HCI データパスの codec ネゴがハングした履歴。T3 で SCO が張れない場合は T13 の btmon ログを添えて切り分け（`esp32-debug-guide.pdf` §9）。
- **ISS-015**：I2S 8 kHz と SCO クロックの独立ドリフト。T6 は必ず 30 分フルで実施し、断の発生が時間比例かを見る。
- **ISS-016**：通話偽装（`call=1`）で SRL-3 が「通話中」表示になり副作用が出る可能性。T3/T4 中に SRL-3 の挙動（アナウンス、他機能の抑制）を観察・記録。副作用が大きければ簡略版へ切替（設計のコメント参照）。
- **ISS-017**：PCM5102A の 8 kHz 再構成。T9 で高域のエイリアス/ノイズが気になる場合は Phase 2 の 8k→48k 補間を検討。
- Pixel でのペアリング後は NVS のピア枠が Pixel で埋まる。T1/T2 の後は必ず BOND-CLR してから B/C の試験へ。

7 コマンド早見表

```
# ログ
idf.py -p /dev/ttyUSB0 monitor | tee qa_$(date +%Y%m%d_%H%M).log
sudo btmon | tee qa_hci.log

# リグ A 接続
bluetoothctl        # scan on / pair / trust / connect <MAC>
pactl set-card-profile bluez_card.<MAC> headset-audio-gateway
pactl list short sinks | grep bluez

# 録音・測定
arecord -D hw:1,0 -f S16_LE -r 48000 -c 1 -d 60 out.wav
ffmpeg -i out.wav -af volumedetect -f null -
ffmpeg -i out.wav -af silencedetect=n=-50dB:d=0.2 -f null -
ffmpeg -i out.wav -lavfi showspectrumpic=s=1024x512 spec.png

# 後始末
bluetoothctl remove <MAC>          # PC / Pixel 側
# ESP32: BOND-CLR (GPIO14) 3 秒長押し
```

基準：`dummy_intercom_spec.md` rev 0.6 §6（オーディオ仕様）/ §11（テスト計画）/ §12（リスク）、`docs/agent_team/test_report.md`、`docs/agent_team/issue_list.md`。 合格基準の数値（遅延 < 100 ms、LINE 0.3–1.0 Vrms / MIC 3–30 mVrms、ノイズ -60 dBFS、断 3 秒）は仕様書 §11 準拠。 関連：`esp32-flashing-guide.pdf`（書き込み）、`esp32-pcm5102a-wiring.pdf`（配線）、`esp32-debug-guide.pdf`（デバッグ）。